# LAPORAN AKHIR PRAKTIKUM M-1

### Laboratorium Sistem Informasi Kalimalang J1525

Nama: Dicky Asqaeliany Ibnul Hakim
Kelas: 2KB04
NPM: 20125258

## 1. Tujuan Praktikum

Membuat program Java sederhana bertema Sistem Informasi Hewan dengan menerapkan konsep Object-Oriented Programming (OOP): encapsulation, inheritance, dan polymorphism.

- Memahami encapsulation melalui access modifier private + getter/setter.
- Memahami inheritance melalui subclass Kucing dan Sapi yang extends Hewan.
- Memahami polymorphism melalui overriding method bersuara().
- Mampu membuat objek, memberi nama, dan menampilkan informasi hewan via main().

## 2. Source Code Program

Seluruh file berada dalam satu folder p1/ tanpa package agar mudah di-compile. Nama file wajib sama dengan nama class.

**File: p1/Hewan.java**

```
// Parent class untuk semua hewan
public class Hewan {
    // Encapsulation: atribut private, hanya bisa diakses via
getter/setter
    private String nama;

    // Constructor kosong
    public Hewan() {
    }

    // Constructor dengan parameter nama
    public Hewan(String nama) {
        this.nama = nama;
    }
```

```
    // Getter untuk membaca nama
    public String getNama() {
        return nama;
    }

    // Setter untuk mengubah nama
    public void setNama(String nama) {
        this.nama = nama;
    }

    // Method yang akan di-override oleh subclass
    public void bersuara() {
        System.out.println("Suara: Hewan bersuara...");
    }
}
```

**File: p1/Kucing.java**

```
// Subclass Kucing mewarisi Hewan
public class Kucing extends Hewan {

    public Kucing() {
        super();
    }

    public Kucing(String nama) {
        super(nama);
    }

    // Polymorphism: override perilaku bersuara() khas kucing
    @Override
    public void bersuara() {
        System.out.println("Suara: Meong!");
    }
}
```

**File: p1/Sapi.java**

```java
// Subclass Sapi mewarisi Hewan
public class Sapi extends Hewan {

    public Sapi() {
        super();
    }

    public Sapi(String nama) {
        super(nama);
    }

    // Polymorphism: override perilaku bersuara() khas sapi
    @Override
    public void bersuara() {
        System.out.println("Suara: Mooo!");
    }
}
```

**File: p1/Main.java**

```java
// Program utama: membuat objek dan menampilkan informasi hewan
public class Main {
    public static void main(String[] args) {
        System.out.println("=== Informasi Hewan ===");
        System.out.println();

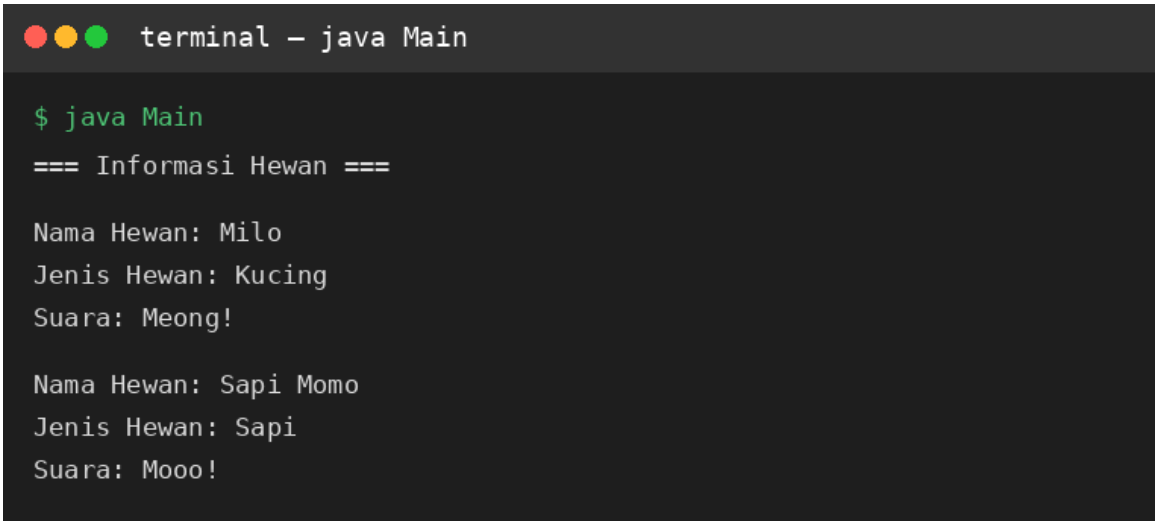
        // Objek 1: Kucing bernama Milo
        Kucing kucing = new Kucing();
        kucing.setNama("Milo");
        System.out.println("Nama Hewan: " + kucing.getNama());
        System.out.println("Jenis Hewan: Kucing");
        kucing.bersuara();
        System.out.println();

        // Objek 2: Sapi bernama Sapi Momo
        Sapi sapi = new Sapi();
        sapi.setNama("Sapi Momo");
        System.out.println("Nama Hewan: " + sapi.getNama());
        System.out.println("Jenis Hewan: Sapi");
```

```
        sapi.bersuara();
    }
}
```

## 3. Hasil Eksekusi Program (Screenshot + Output)

Screenshot terminal hasil perintah java Main




## 4. Penjelasan Alur Logika Program

Program berjalan sekuensial dari method main() di Main.java. Tidak ada input pengguna; nama hewan di-hardcode agar output sama dengan contoh soal.

1. Cetak judul '=== Informasi Hewan ==='.

2. Buat objek Kucing via 'new Kucing()'. Objek ini mewarisi atribut nama dari Hewan.

3. Isi nama dengan kucing.setNama("Milo") — bukti encapsulation (akses via setter, bukan langsung).

4. Tampilkan 'Nama Hewan: ' + kucing.getNama() (akses via getter) dan teks tetap 'Jenis Hewan: Kucing'.

5. Panggil kucing.bersuara(). Karena Kucing meng-override method ini, yang tercetak 'Suara: Meong!' (polymorphism).

6. Ulangi langkah 2–5 untuk Sapi: new Sapi(), setNama("Sapi Momo"), tampilkan, panggil sapi.bersuara() -> 'Suara: Mooo!'.

7. Program selesai. Java memilih versi bersuara() berdasarkan tipe objek asli, bukan tipe parent.

Kenapa output suara bisa beda padahal nama method sama? Karena polymorphism + overriding: satu tanda tangan method bersuara(), dua perilaku berbeda di Kucing dan Sapi.

## 5. Kesimpulan

1. Encapsulation: atribut nama yang private terbukti aman — hanya bisa dibaca/diubah lewat getNama()/setNama(). Jika atribut public, data bisa diubah sembarangan dari luar class.
2. Inheritance: Kucing dan Sapi tidak perlu menulis ulang atribut nama dan getter/setter karena mewarisi dari Hewan (extends). Ini menghemat kode dan memudahkan perawatan.
3. Polymorphism: method bersuara() yang sama menghasilkan 'Meong!' untuk kucing dan 'Mooo!' untuk sapi berkat @Override. Kode di Main tetap sederhana: cukup panggil bersuara() tanpa if-else jenis hewan.
4. Secara keseluruhan, praktikum ini membuktikan 4 file kecil sudah cukup menunjukkan 3 pilar OOP dan menghasilkan output persis seperti contoh soal.